25. Graphs: BFS, DFS, Shortest Paths Intro

25. Graphs: BFS, DFS, Shortest Paths Intro

Phase: Phase 4 — Algorithms and data structures
Estimated time: 2 days
Slug: `25-graphs`

Learning objectives

- Represent graphs with adjacency lists
- Implement BFS and DFS
- Detect cycles in directed graphs
- Introduce Dijkstra at a conceptual+code level

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Representation

```
type Graph map[string][]string // adjacency list
```

Prefer adjacency lists for sparse graphs (most real networks).

BFS / DFS

- BFS: queue — shortest path in unweighted graphs
- DFS: stack/recursion — cycle detection, topological vibes

```
func BFS(g Graph, start string) []string {
    seen := map[string]bool{start: true}
    q := []string{start}
    var order []string
    for len(q) > 0 {
        u := q[0]; q = q[1:]
        order = append(order, u)
        for _, v := range g[u] {
            if !seen[v] { seen[v] = true; q = append(q, v) }
        }
    }
    return order
}
```

Weighted shortest paths

Dijkstra with a priority queue finds shortest paths for non-negative weights — core infra for routing and dependency costs.

Common mistakes (especially from Python instincts)

- Adjacency matrix for huge sparse graphs

- Forgetting seen set → infinite loops
- Dijkstra with negative edges

Exercises

Work in `exercises/25-graphs/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/25-graphs/` until you have an honest failing attempt.

Self-check

1. When BFS vs DFS?
2. Why adjacency list?
3. Dijkstra precondition?

Further reading

- https://en.wikipedia.org/wiki/Graph_traversal